NAME : Rithish Pm

SECTION : G

SRN : PES2UG24CS405

SUB : OS (Orange)

Github link : https://github.com/writztheritz/PES2UG24CS405-pes-vcs

Phase 1 A

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Phase 1 B

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2 A

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Phase 2 B

Phase 3 A

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ make pes
make: 'pes' is up to date.
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ echo "hello" > file1.txt
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ echo "world" > file2.txt
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ ./pes add file1.txt file2.txt
```

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  tree.h
  untracked:  commit.c
  untracked:  test_tree
  untracked:  pes.h
  untracked:  test_objects
  untracked:  hello.txt
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  commit.h
  untracked:  test_tree.c
```

Phase 3 B

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776757145 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776757148 6 file2.txt
```

Phase 4 A

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ ./pes log
commit 47d12e5f2f90e426553982e0a316aadc844e1ca6ea28d38ea2cdf93bcd97bf02
Author: PES User <pes@localhost>
Date:   1776757700

    Add farewell

commit 3d72b766e3bfcbff0aa8b1f292d4bae91cbeb9464fa52098c645bcfe66eb7832
Author: PES User <pes@localhost>
Date:   1776757684

    Add worldell

commit 96ad6c43b2f16f0e10279847f10287987249febc0c33f2f386a2a811b24d286b
Author: PES User <pes@localhost>
Date:   1776757667

    Initial committial commiQ
```

Phase 4 B

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/3d/72b766e3bfcbff0aa8b1f292d4bae91cbeb9464fa52098c645bcfe66eb7832
.pes/objects/47/d12e5f2f90e426553982e0a316aadc844e1ca6ea28d38ea2cdf93bcd97bf02
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/96/ad6c43b2f16f0e10279847f10287987249febc0c33f2f386a2a811b24d286b
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Phase 4 C

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ cat .pes/refs/heads/main
47d12e5f2f90e426553982e0a316aadc844e1ca6ea28d38ea2cdf93bcd97bf02
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final integration test

```
vboxuser@ubuntu:~/Desktop/PES2UG24CS405-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 47705d9671df... Initial commit
```

```
Log after first commit:
commit 47705d9671dff986eb4913afd34d7dab47af9357a1d16f2b0ac84905c5ee8a48
Author: PES User <pes@localhost>
Date:   1776757974

    Initial commit


--- Second Commit ---
Committed: 80ba63862a86... Update file.txt

--- Third Commit ---
Committed: 55c1c20d54d7... Add farewell

--- Full History ---
commit 55c1c20d54d773e2e60c7965ff7c1dabd137365fa2803719db1d75726bc78ee1
Author: PES User <pes@localhost>
Date:   1776757974

    Add farewell

commit 80ba63862a863d3a5ed33c606ee97cf6903e78ec09b5e25215f8aa2207ca3f9a
Author: PES User <pes@localhost>
Date:   1776757974

    Update file.txt
```

```
commit 47705d9671dff986eb4913afd34d7dab47af9357a1d16f2b0ac84905c5ee8a48
Author: PES User <pes@localhost>
Date:   1776757974

    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
55c1c20d54d773e2e60c7965ff7c1dabd137365fa2803719db1d75726bc78ee1

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/47/705d9671dff986eb4913afd34d7dab47af9357a1d16f2b0ac84905c5ee8a48
.pes/objects/55/c1c20d54d773e2e60c7965ff7c1dabd137365fa2803719db1d75726bc78ee1
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/80/ba63862a863d3a5ed33c606ee97cf6903e78ec09b5e25215f8aa2207ca3f9a
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Phase 5

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

> In PES-VCS, pes checkout <branch> would first read the commit hash stored in .pes/refs/heads/<branch> and update .pes/HEAD to point to that branch. Then it must load the target commit's tree, update the working directory files to match that snapshot, remove files no longer present, and recreate the index accordingly. This operation is complex because it changes HEAD, index, and actual files together. It must also avoid overwriting user changes or deleting important data accidentally.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

> To detect a dirty working directory, compare each tracked file in the index with the current working file. If size or modification time differs, recompute the file's hash and compare it with the hash stored in .pes/index. If they differ, the file has uncommitted changes. Then compare that file with the target branch's version from the object store. If checkout would overwrite those edits, the command must refuse and warn the user.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

> In detached HEAD state, .pes/HEAD stores a commit hash directly instead of a branch reference. New commits can still be created, but no branch name moves forward to track them. If the user switches branches later, those commits may become unreachable. Recovery is possible by creating a new branch pointing to those commits or using commit hashes from logs/reflog-like history. Detached HEAD is useful for temporary experiments.

Phase 6

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

> Garbage collection would start from all branch heads in .pes/refs/heads/ and mark every reachable commit, tree, and blob by recursively following parent links and tree references. After marking, any object file not visited is unreachable and can be deleted. A hash set is ideal for storing visited hashes efficiently. For 100,000 commits and 50 branches, roughly all reachable commits plus their trees and blobs must be scanned, so several hundred thousand objects may be visited.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

> Running garbage collection during a commit is dangerous because GC may delete objects created by the commit before the branch reference is updated. For example, a tree object may be written, but before the new commit points to it, GC sees no reference and removes it. Then the commit finishes with missing objects, corrupting history. Real Git avoids this using locks, temporary files, and only deleting objects older than a safety grace period.